Name: Vũ Hùng Phi

StudentID: 524K0008

Report-Lab09

I. Introduction

This lab focuses on unit testing and automation testing using C# .NET Core in Visual Studio. The main purpose is to understand how to create and execute unit tests using different testing frameworks such as MSTest, xUnit, and NUnit. Additionally, the lab introduces automation testing with Selenium WebDriver and data-driven testing using CSV files.

Through these exercises, students learn how to ensure software correctness, improve test coverage, and automate repetitive testing tasks efficiently.

II. Exercise

1. Exercise 1

Description

In this exercise, a console application named *BasicMath* was created. A class called BasicMaths was implemented with four basic operations: Add, Subtract, Multiply, and Divide. A unit test project was then created to verify these methods using MSTest and xUnit.

Steps

1. Created a Console App project named *BasicMath*.
2. Added a class BasicMaths with mathematical methods.
3. Created a Unit Test Project named *BasicMathTests*.
4. Added a reference from the test project to the main project.
5. Implemented unit tests using [DataTestMethod] and [DataRow].
6. Executed tests using Test Explorer.

What I learned

- How to create unit tests in Visual Studio
- How to use MSTest and xUnit frameworks
- How to apply Equivalence Partitioning (EP) and Boundary Value Analysis (BVA)
- How to validate expected and actual results

2. Exercise 2

Description

This exercise focuses on data-driven testing using CSV files. Test data was stored externally and loaded into test methods using CsvHelper.

Steps

1. Created CSV files for each operation (add, subtract, multiply, divide).
2. Installed CsvHelper library.
3. Implemented a method to read CSV data.
4. Used [MemberData] to pass data into test methods.
5. Executed tests with multiple datasets.

What I learned

- How to perform data-driven testing
- How to read external data using CSV files
- How to separate test logic from test data
- How to improve scalability of test cases

3. Exercise 3

Description

A BankAccount system was implemented with operations such as Debit, Credit, and Withdraw. Unit tests were created to validate both normal and exceptional cases.

Steps

1. Created a console project *BankAccountApp*.
2. Implemented BankAccount class with required methods.
3. Created a test project *BankAccountTests*.
4. Wrote unit tests using [Theory] and [InlineData].
5. Implemented exception handling for insufficient funds.
6. Extended testing using CSV data.

What I learned

- How to test real-world scenarios
- How to handle exceptions in unit testing
- How to design test cases using EP and BVA
- How to validate both valid and invalid inputs

4. Exercise 4

Description

This exercise introduces automation testing using Selenium WebDriver. A login page was created and automated tests were written to simulate user login using data from CSV files.

Steps

1. Created a simple HTML login page.
2. Created a CSV file containing login credentials.
3. Created a Class Library project *LoginAutomation*.
4. Installed Selenium WebDriver and CsvHelper.
5. Implemented automated test script using Selenium.
6. Executed tests through Visual Studio Test Explorer.

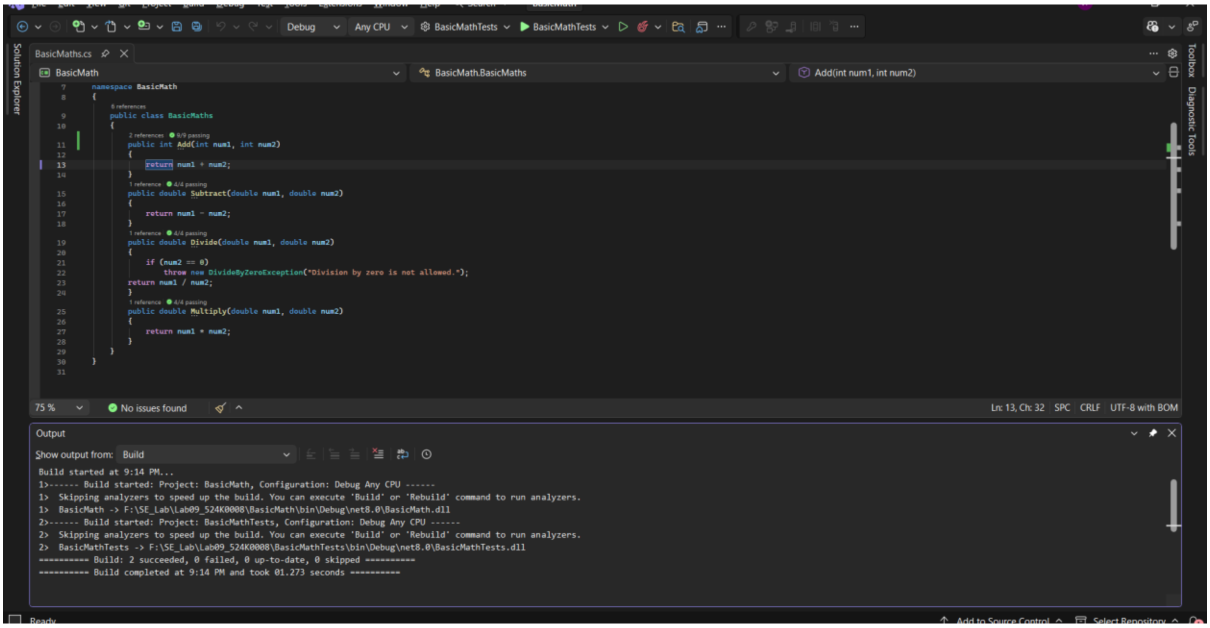
What I learned

- How to automate browser actions using Selenium
- How to interact with web elements (input, button)
- How to perform data-driven automation testing
- How to simulate real user behavior

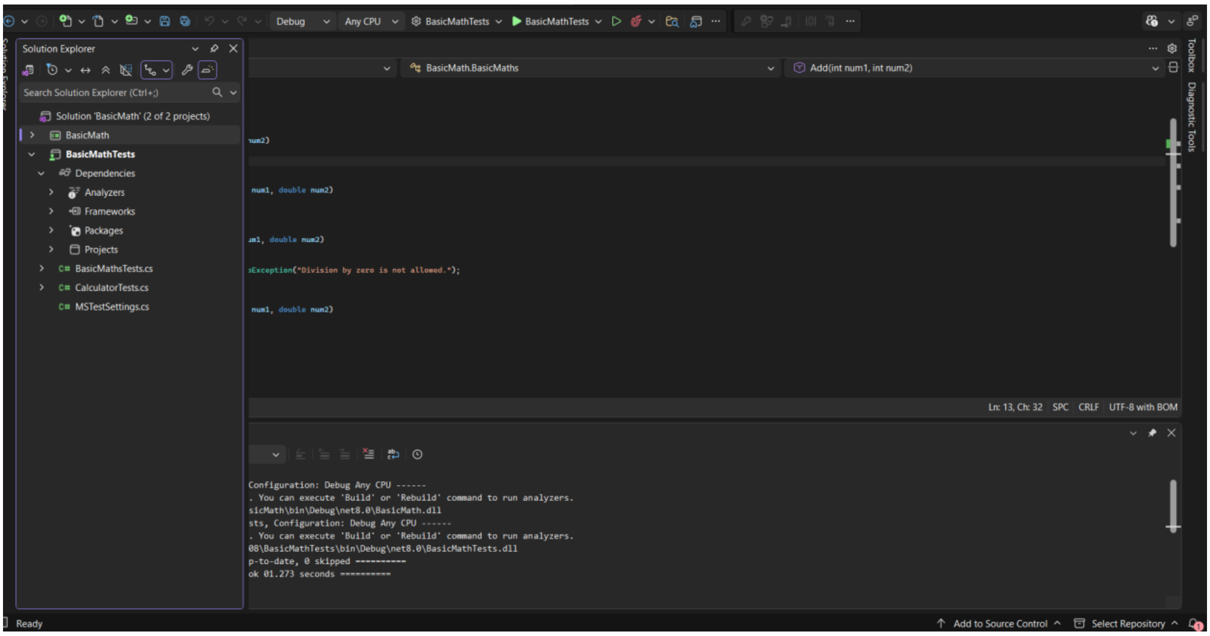
III. Screenshots

1. Exercise 1

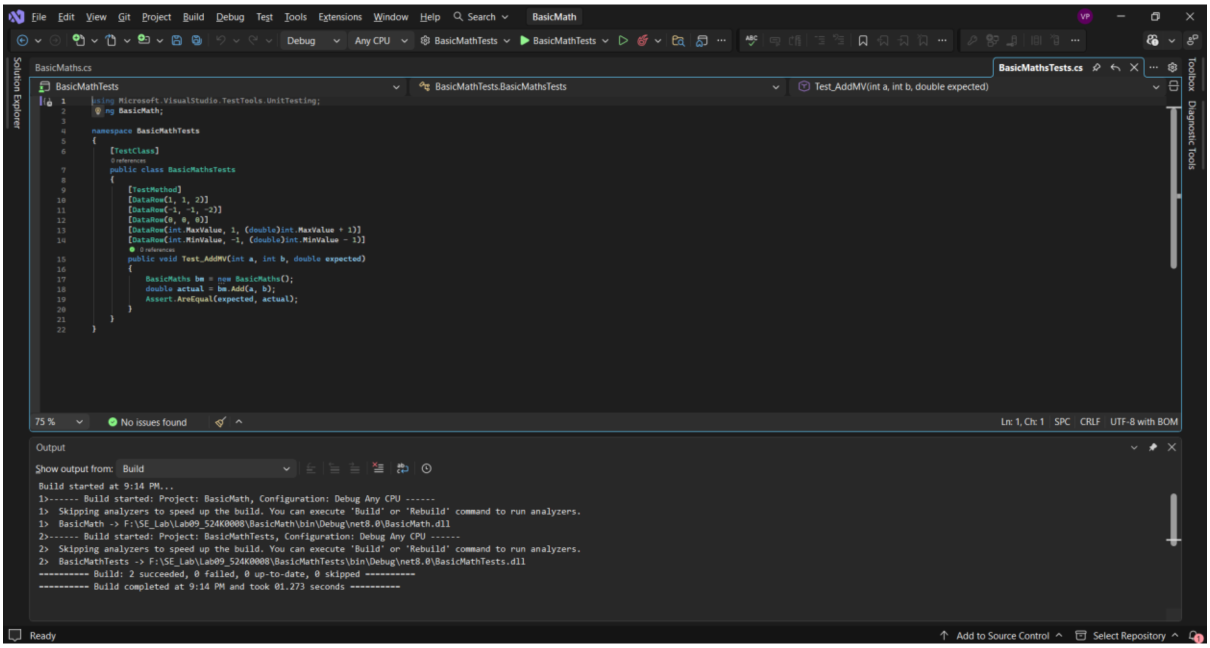
Step 1



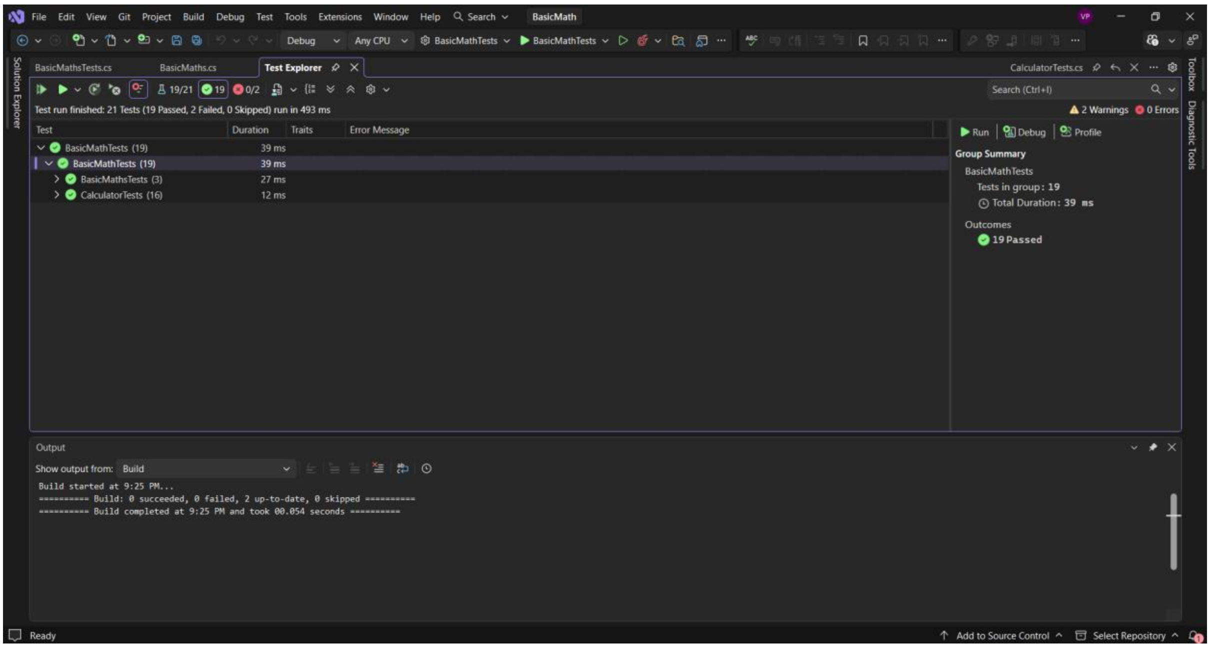
Step 2



Step 3

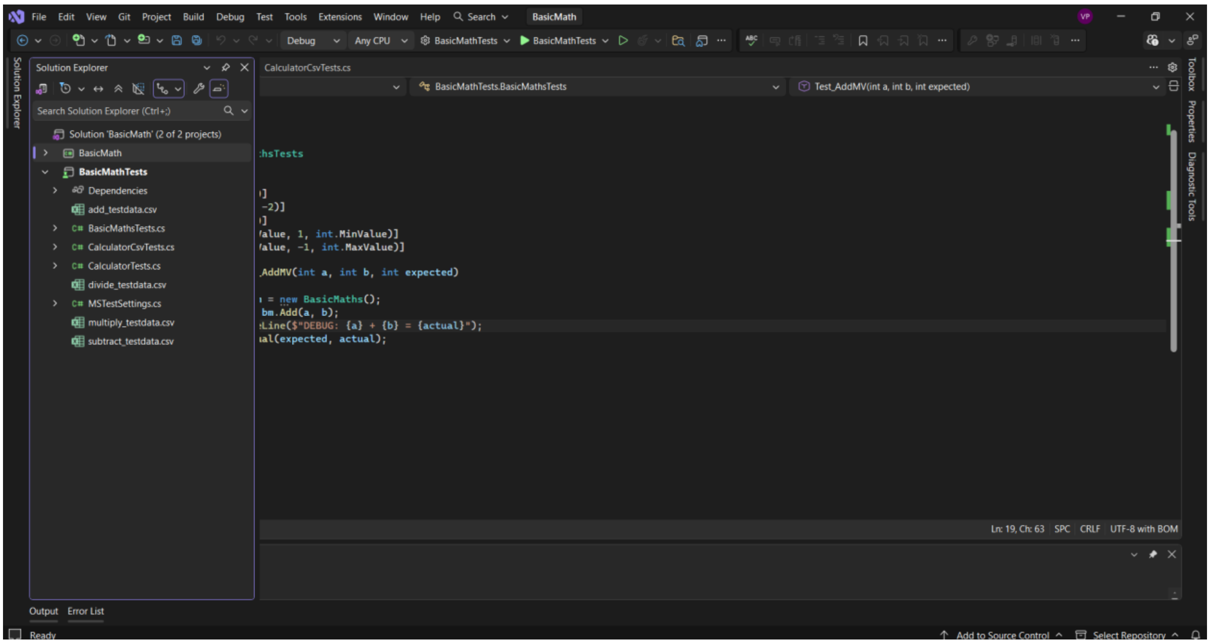


Test

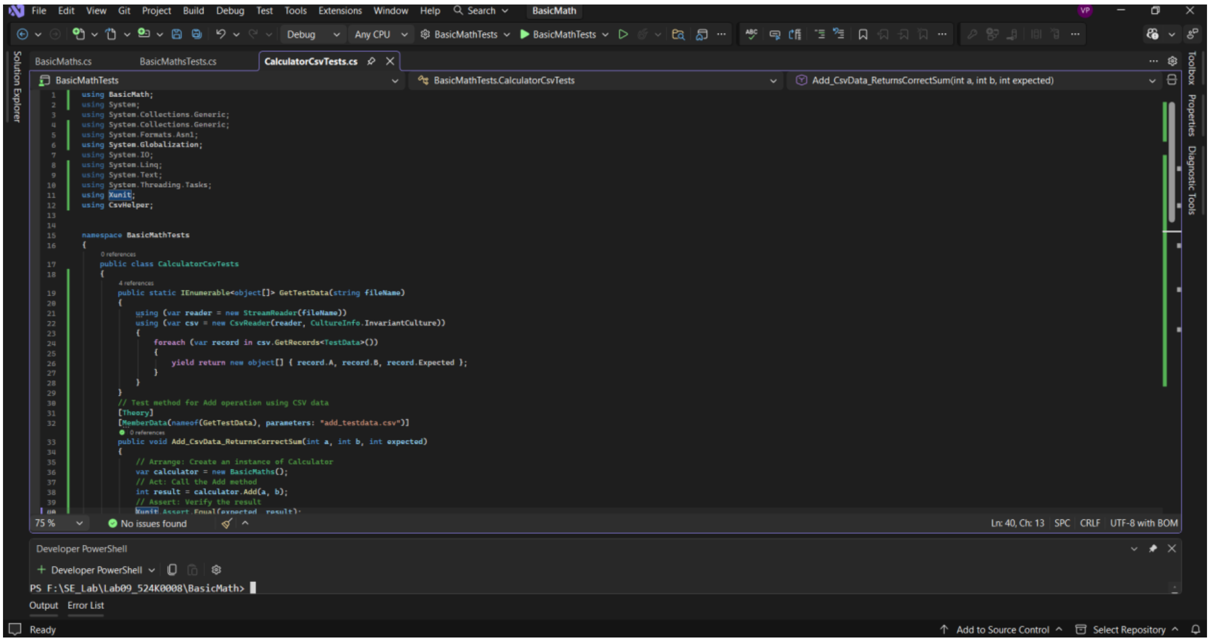


2. Exercise 2

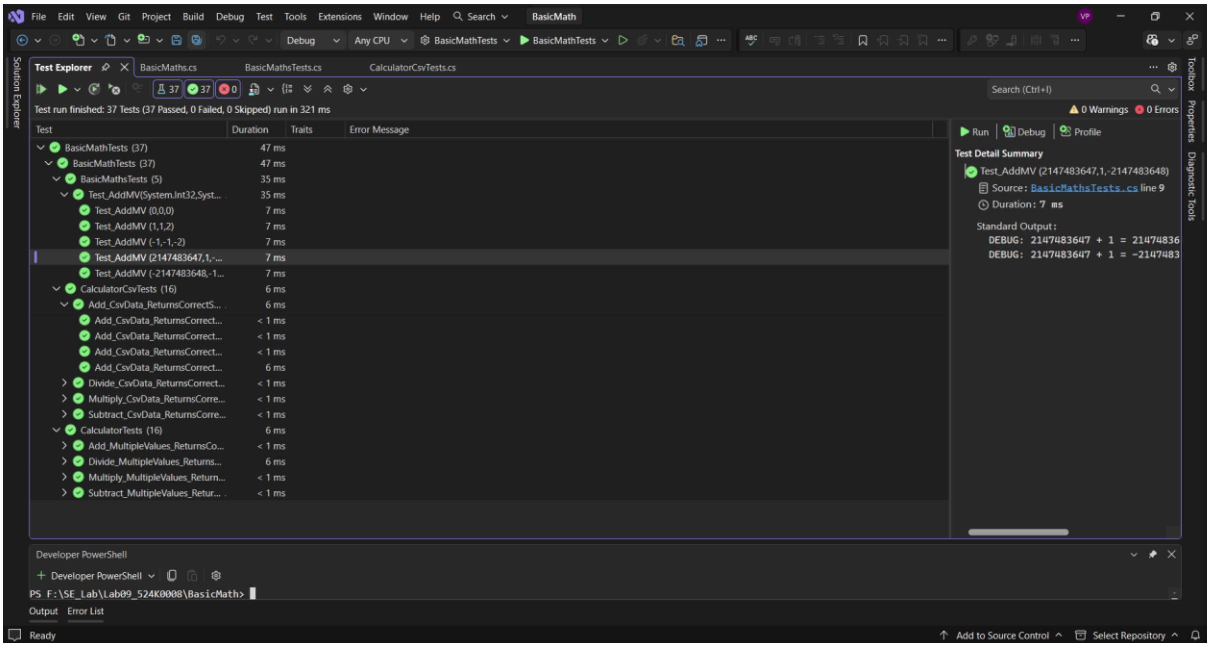
Step 1


Solution Explorer
Solution 'BasicMath' (2 of 2 projects)
BasicMath
BasicMathTests
Dependencies
add_testdata.csv
BasicMathsTests.cs
CalculatorCsvTests.cs
CalculatorTests.cs
divide_testdata.csv
MSTestSettings.cs
multiply_testdata.csv
subtract_testdata.csv
CalculatorCsvTests.cs
BasicMathTests.BasicMathsTests
Test_AddMV(int a, int b, int expected)
Ln: 19, Ch: 63 SPC CRLF UTF-8 with BOM
Output Error List
Ready
Add to Source Control
Select Repository

Step 2

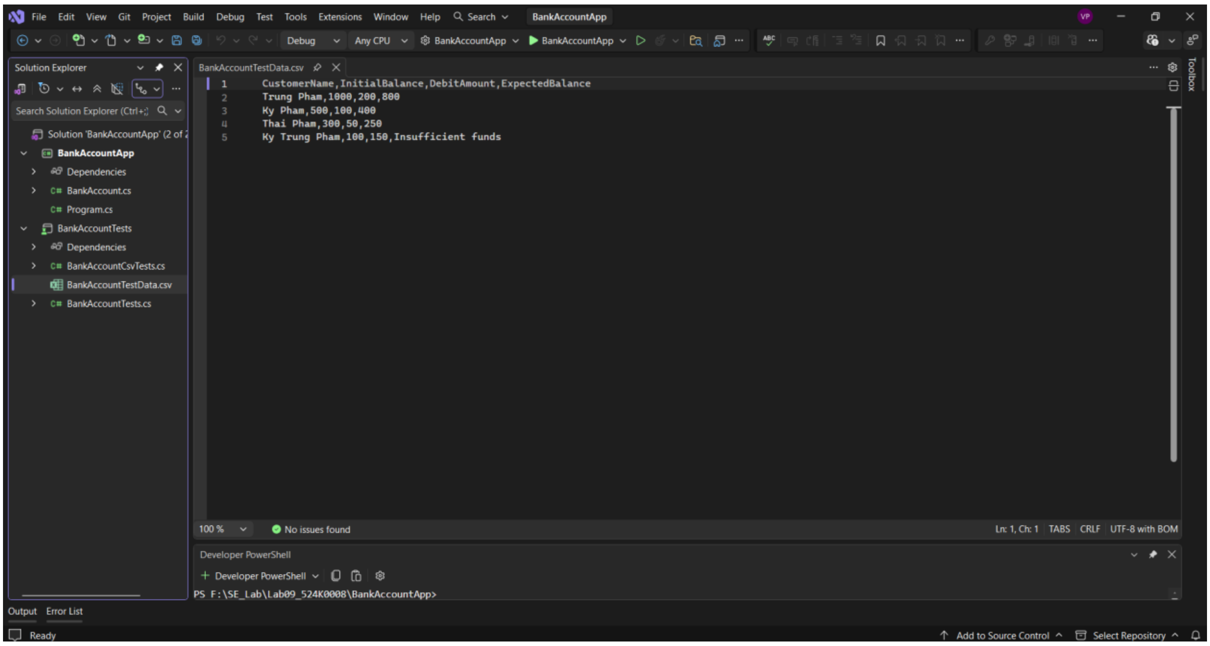

BasicMaths.cs
BasicMathsTests.cs
CalculatorCsvTests.cs
BasicMathTests
BasicMathTests.CalculatorCsvTests
Add_CsvData_ReturnsCorrectSum(int a, int b, int expected)
using BasicMath;
using System;
using System.Collections.Generic;
using System.Collections.Generic;
using System.Formats.Asn1;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xunit;
using CsvHelper;
namespace BasicMathTests
{
public class CalculatorCsvTests
{
public static IEnumerable<object[]> GetTestData(string fileName)
{
using (var reader = new StreamReader(fileName))
using (var csv = new CsvReader(reader, CultureInfo.InvariantCulture))
{
foreach (var record in csv.GetRecords<TestData>())
{
yield return new object[] { record.A, record.B, record.Expected };
}
}
}
// Test method for Add operation using CSV data
[Theory]
[MemberData(nameof(GetTestData), parameters: "add_testdata.csv")]
public void Add_CsvData_ReturnsCorrectSum(int a, int b, int expected)
{
// Arrange: Create an instance of Calculator
var calculator = new BasicMaths();
// Act: Call the Add method
int result = calculator.Add(a, b);
// Assert: Verify the result
75 %
No issues found
Ln: 40, Ch: 13 SPC CRLF UTF-8 with BOM
Developer PowerShell
PS F:\SE_Lab\Lab09_524K0008\BasicMath>
Output Error List
Ready
Add to Source Control
Select Repository

Step 3

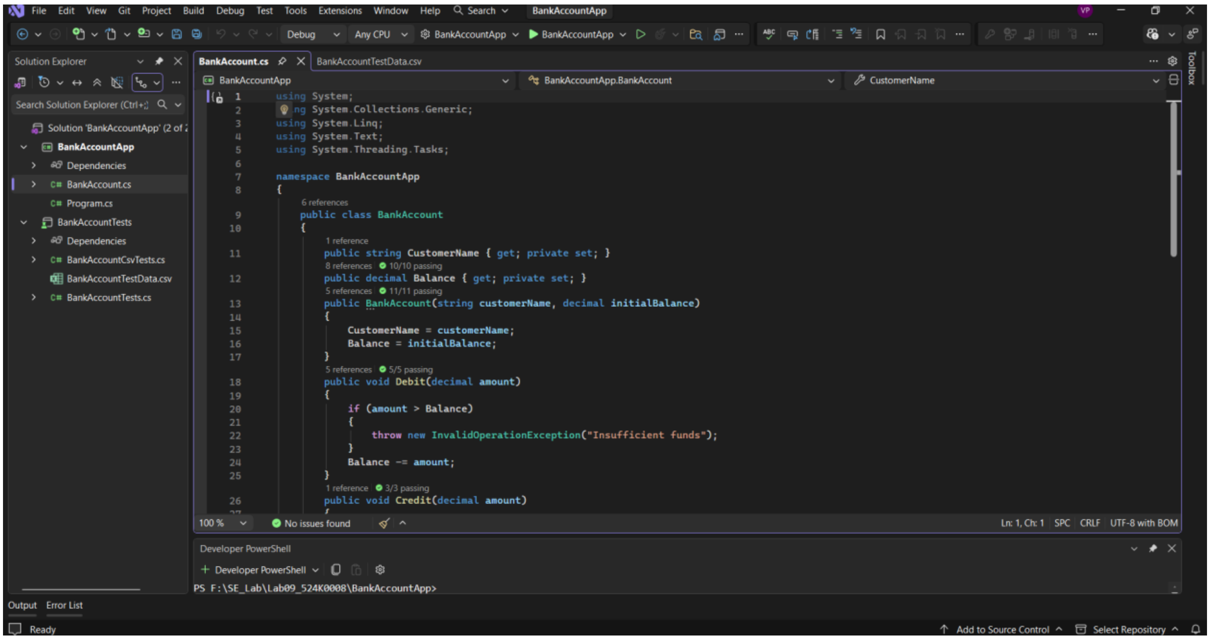


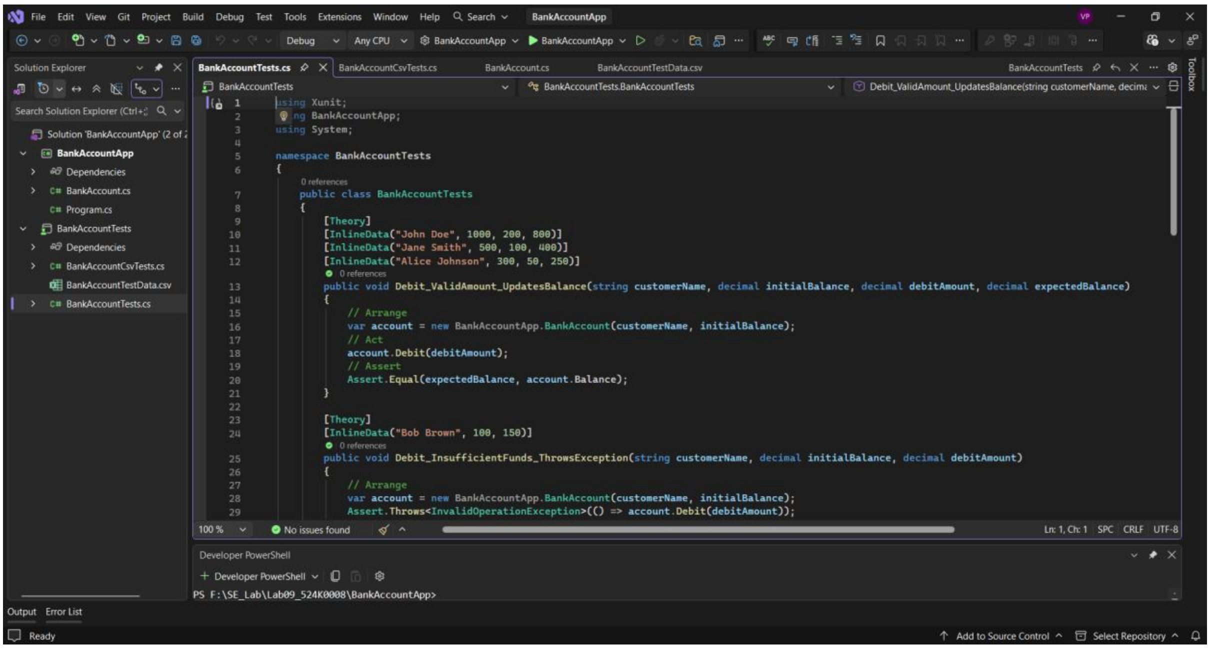
3. Exercise 3

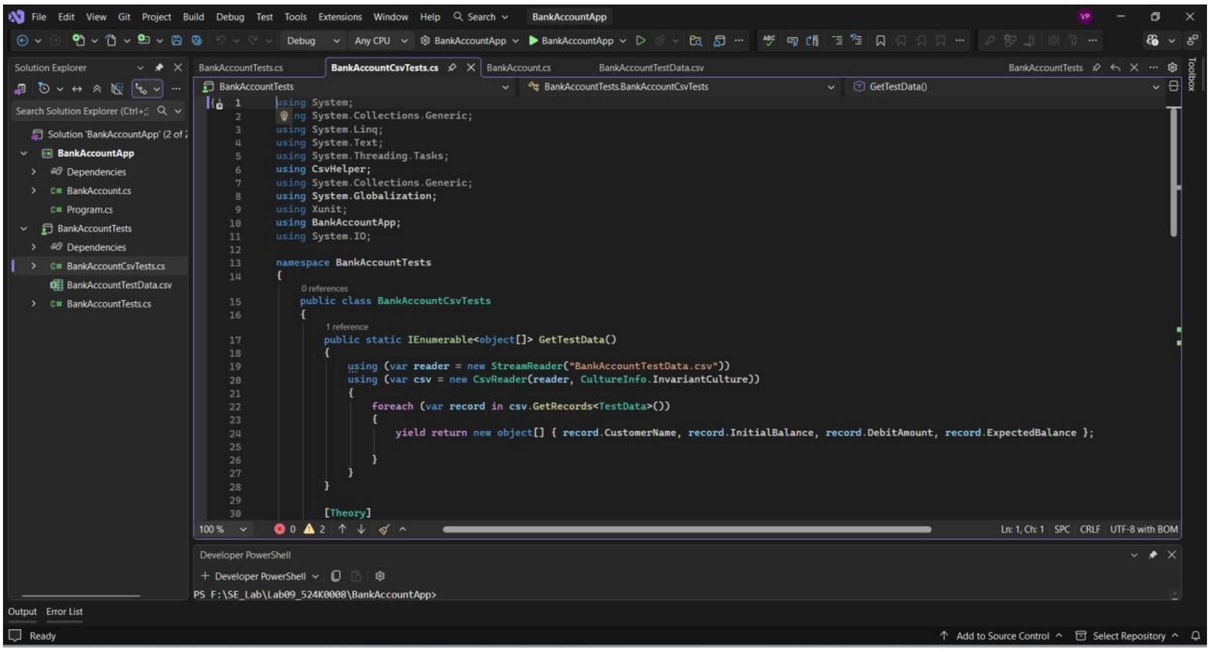
Step 1



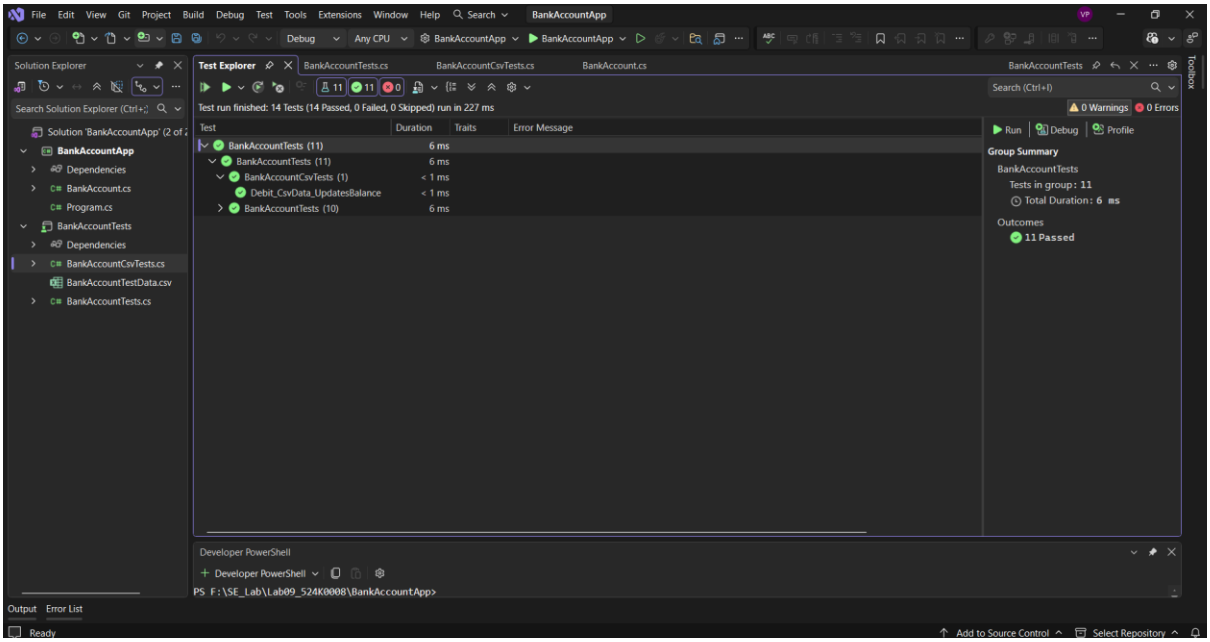
Step 2


BankAccount.cs
BankAccountTestData.csv
Solution Explorer
BankAccountApp
Dependencies
BankAccount.cs
Program.cs
BankAccountTests
Dependencies
BankAccountCsvTests.cs
BankAccountTestData.csv
BankAccountTests.cs
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace BankAccountApp
{
public class BankAccount
{
public string CustomerName { get; private set; }
public decimal Balance { get; private set; }
public BankAccount(string customerName, decimal initialBalance)
{
CustomerName = customerName;
Balance = initialBalance;
}
public void Debit(decimal amount)
{
if (amount > Balance)
{
throw new InvalidOperationException("Insufficient funds");
}
Balance -= amount;
}
public void Credit(decimal amount)
Developer PowerShell
PS F:\SE_Lab\Lab09_524K0008\BankAccountApp>


BankAccountTests.cs
BankAccountCsvTests.cs
BankAccount.cs
BankAccountTestData.csv
using Xunit;
using BankAccountApp;
using System;
namespace BankAccountTests
{
public class BankAccountTests
{
[Theory]
[InlineData("John Doe", 1000, 200, 800)]
[InlineData("Jane Smith", 500, 100, 400)]
[InlineData("Alice Johnson", 300, 50, 250)]
public void Debit_ValidAmount_UpdatesBalance(string customerName, decimal initialBalance, decimal debitAmount, decimal expectedBalance)
{
// Arrange
var account = new BankAccountApp.BankAccount(customerName, initialBalance);
// Act
account.Debit(debitAmount);
// Assert
Assert.Equal(expectedBalance, account.Balance);
}
[Theory]
[InlineData("Bob Brown", 100, 150)]
public void Debit_InsufficientFunds_ThrowsException(string customerName, decimal initialBalance, decimal debitAmount)
{
// Arrange
var account = new BankAccountApp.BankAccount(customerName, initialBalance);
Assert.Throws<InvalidOperationException>(() => account.Debit(debitAmount));
Developer PowerShell
PS F:\SE_Lab\Lab09_524K0008\BankAccountApp>

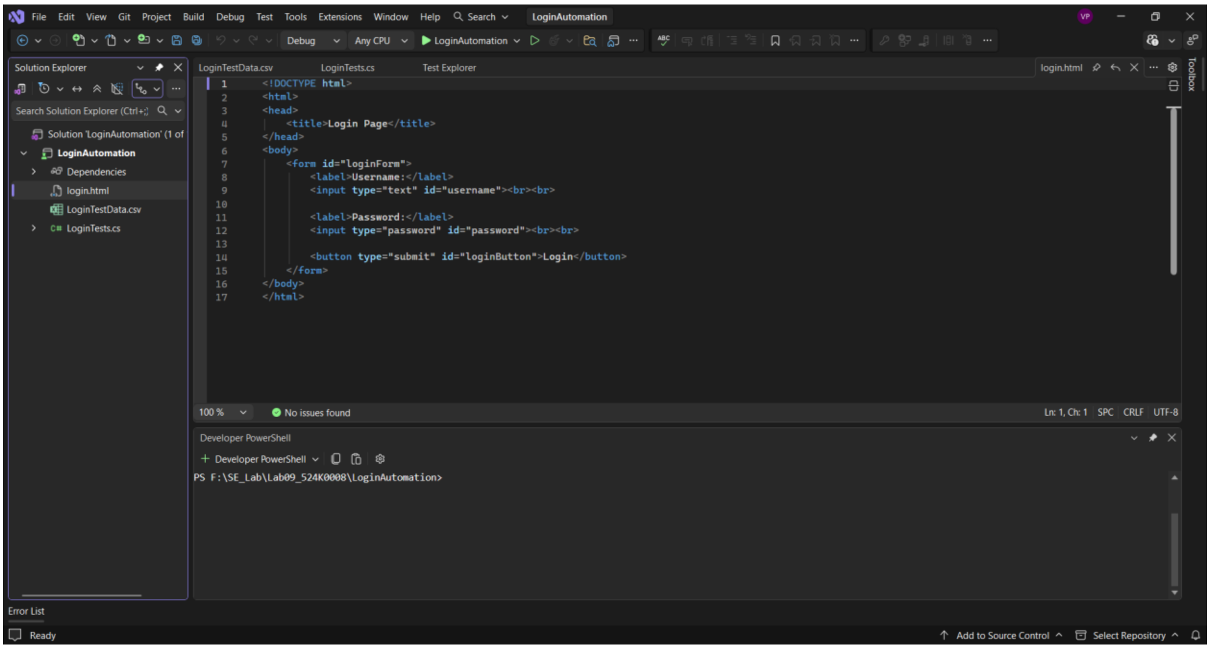


Step 3



4. Exercise 4

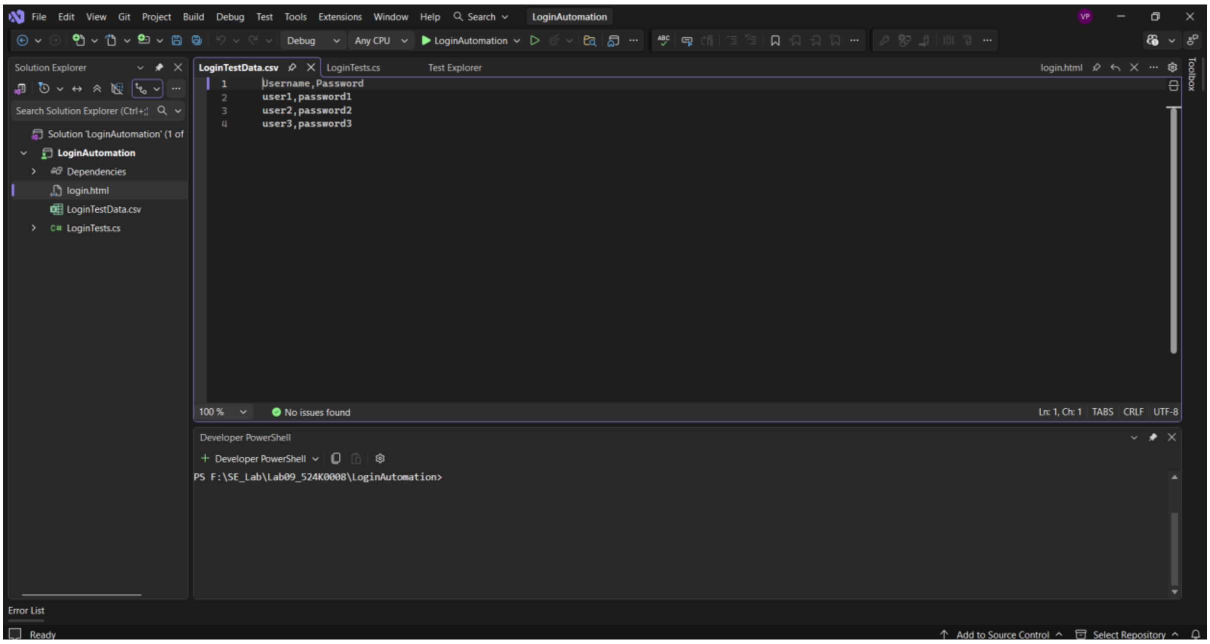
Step 1


LoginAutomation
Solution Explorer
Solution 'LoginAutomation' (1 of
LoginAutomation
Dependencies
login.html
LoginTestData.csv
LoginTests.cs
LoginTestData.csv
LoginTests.cs
Test Explorer
login.html
<!DOCTYPE html>
<html>
<head>
<title>Login Page</title>
</head>
<body>
<form id="loginForm">
<label>Username:</label>
<input type="text" id="username">

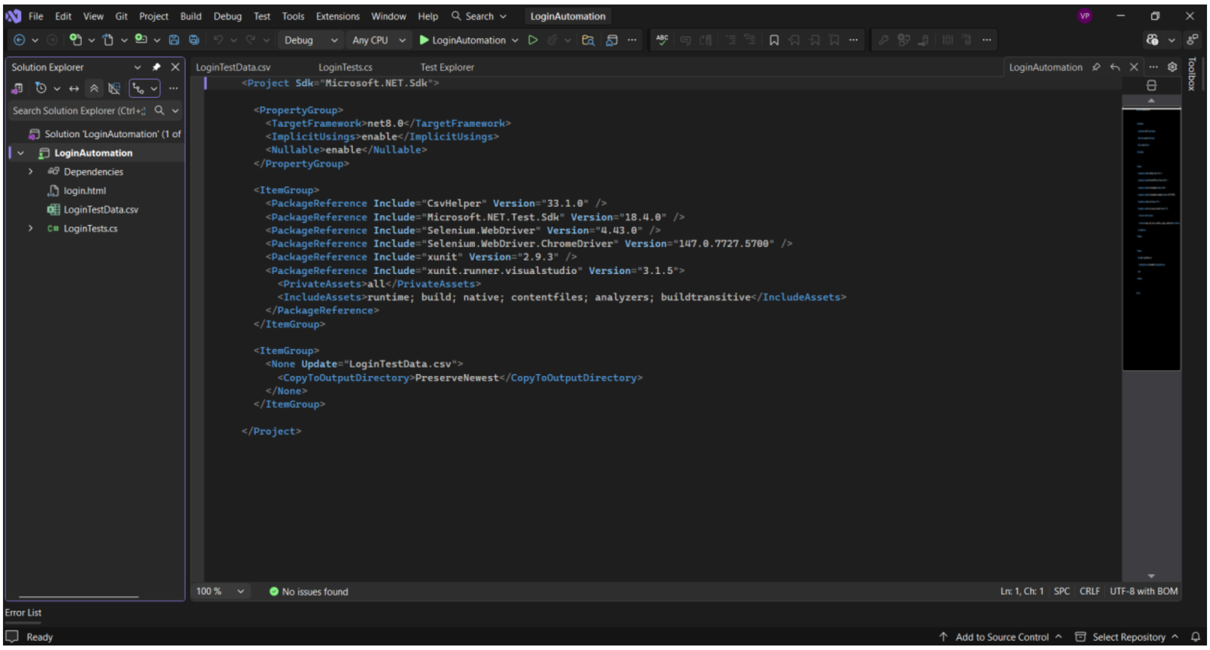
<label>Password:</label>
<input type="password" id="password">

<button type="submit" id="loginButton">Login</button>
</form>
</body>
</html>
No issues found
Developer PowerShell
PS F:\SE_Lab\Lab09_524K0008\LoginAutomation>
Error List
Ready
Add to Source Control
Select Repository

Step 2

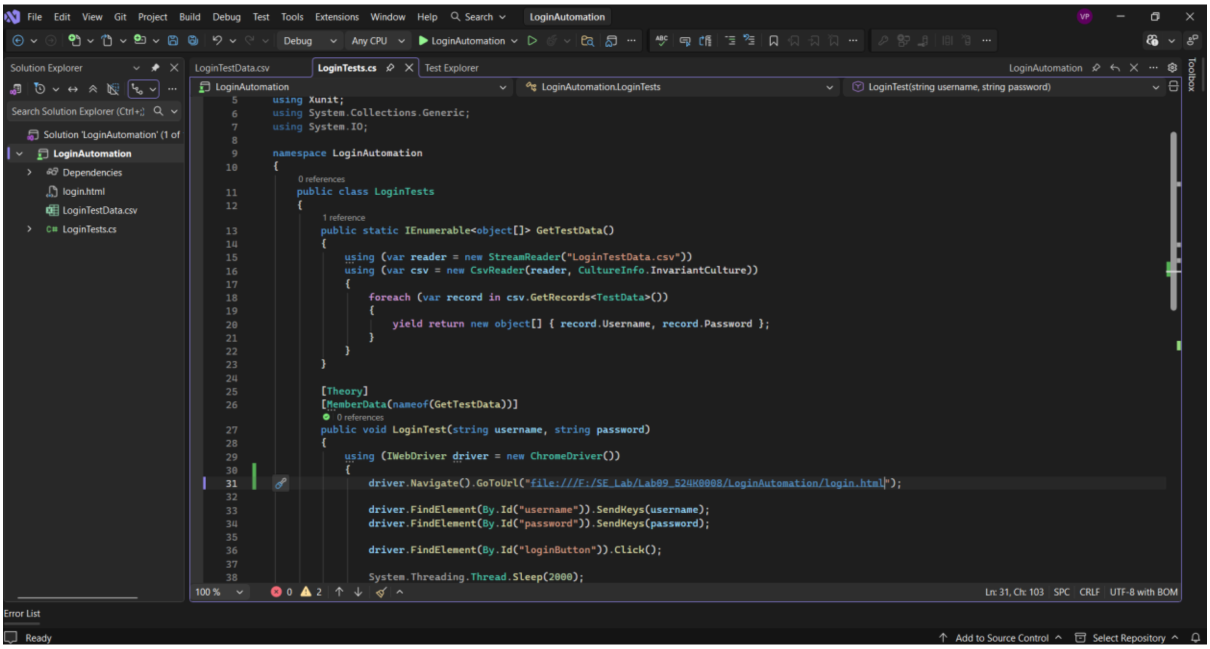

LoginAutomation
Solution Explorer
Solution 'LoginAutomation' (1 of
LoginAutomation
Dependencies
login.html
LoginTestData.csv
LoginTests.cs
LoginTestData.csv
LoginTests.cs
Test Explorer
login.html
Username,Password
user1,password1
user2,password2
user3,password3
No issues found
Developer PowerShell
PS F:\SE_Lab\Lab09_524K0008\LoginAutomation>
Error List
Ready
Add to Source Control
Select Repository

Step 3



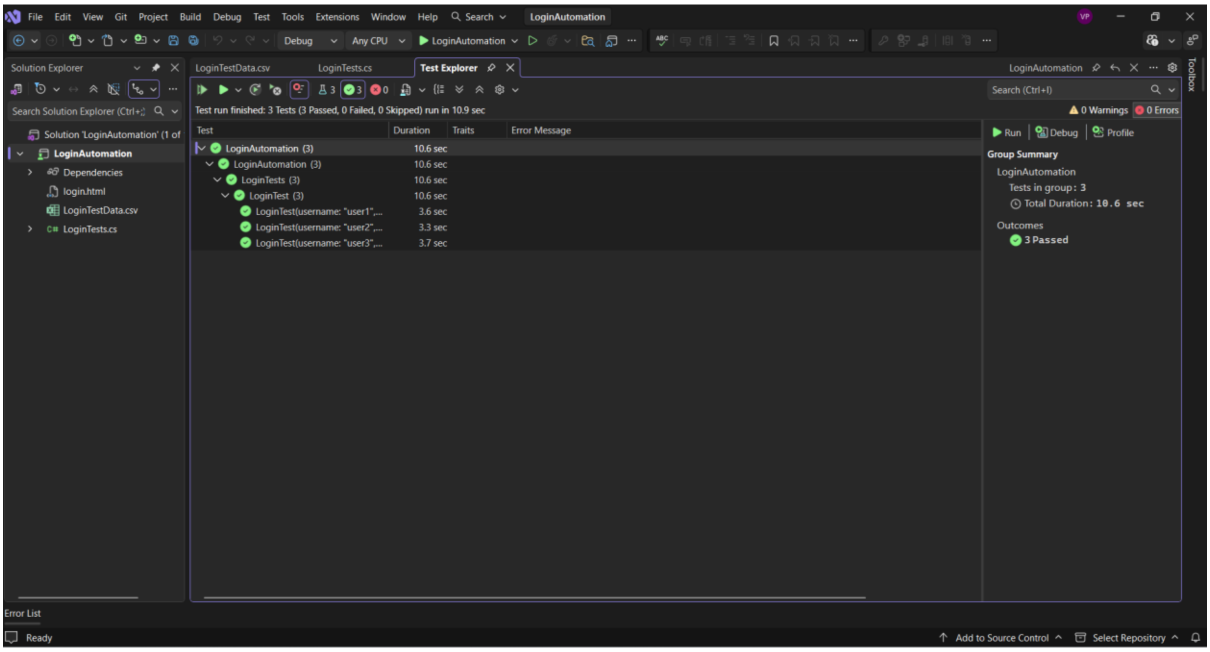
LoginTestData.csv
LoginTests.cs
Test Explorer
Solution Explorer
Solution 'LoginAutomation' (1 of
LoginAutomation
Dependencies
login.html
LoginTestData.csv
LoginTests.cs
<Project Sdk="Microsoft.NET.Sdk">
<PropertyGroup>
<TargetFramework>net8.0</TargetFramework>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
</PropertyGroup>
<ItemGroup>
<PackageReference Include="CsvHelper" Version="33.1.0" />
<PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.4.0" />
<PackageReference Include="Selenium.WebDriver" Version="4.43.0" />
<PackageReference Include="Selenium.WebDriver.ChromeDriver" Version="147.0.7727.5700" />
<PackageReference Include="xunit" Version="2.9.3" />
<PackageReference Include="xunit.runner.visualstudio" Version="3.1.5">
<PrivateAssets>all</PrivateAssets>
<IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
</PackageReference>
</ItemGroup>
<ItemGroup>
<None Update="LoginTestData.csv">
<CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
</None>
</ItemGroup>
</Project>
No issues found
Error List
Ready

Step 4



LoginTestData.csv
LoginTests.cs
Test Explorer
LoginAutomation.LoginTests
LoginTest(string username, string password)
using Xunit;
using System.Collections.Generic;
using System.IO;
namespace LoginAutomation
{
public class LoginTests
{
public static IEnumerable<object[]> GetTestData()
{
using (var reader = new StreamReader("LoginTestData.csv"))
using (var csv = new CsvReader(reader, CultureInfo.InvariantCulture))
{
foreach (var record in csv.GetRecords<TestData>())
{
yield return new object[] { record.Username, record.Password };
}
}
}
[Theory]
[MemberData(nameof(GetTestData))]
public void LoginTest(string username, string password)
{
using (IWebDriver driver = new ChromeDriver())
{
driver.Navigate().GoToUrl("file:///F:/SE_Lab/Lab09_524K0008/LoginAutomation/login.html");
driver.FindElement(By.Id("username")).SendKeys(username);
driver.FindElement(By.Id("password")).SendKeys(password);
driver.FindElement(By.Id("loginButton")).Click();
System.Threading.Thread.Sleep(2000);
Error List
Ready

Step 5



IV. Conclusion

In this lab, I gained practical experience in both unit testing and automation testing using C# .NET Core. I learned how to create and execute test cases using multiple frameworks such as MSTest, xUnit, and NUnit. Additionally, I understood how to perform data-driven testing using CSV files and how to automate web testing using Selenium.

One of the main challenges I faced was configuring test frameworks and resolving errors related to dependencies and file paths, especially when working with CSV files and Allure reporting. However, through debugging and troubleshooting, I was able to resolve these issues and successfully complete all exercises.

Overall, this lab helped me improve my understanding of software testing techniques and tools, which are essential for ensuring software quality in real-world applications.

https://github.com/xiongfei2011/Lab09_524K0008